



Attestry

Fixed-QR Digital Product Passport Platform

"QR 하나로, 이 제품의 모든 진실을 증명한다"

서비스 완성 기준 문서 **v7.0**

일반인부터 개발자까지 — 누구나 이해할 수 있는 완전한 가이드

이런 분이 읽으면
비개발자/일반인
→ 0장·1장만 읽으세요

이런 분이 읽으면
개발자 동료
→ 2~5장 중심으로

이런 분이 읽으면
면접관/투자자
→ 6~7장 원장 시퀀스

0장. 이 서비스가 뭔지 ?

쉬운 설명

"명품 스니커즈를 중고로 살 때, QR 하나만 찍으면 이 제품이 진짜인지, 지금까지 누구를 거쳤는지, 어디서 수선했는지 전부 알 수 있다."

그리고 그 기록은 아무도 지우거나 바꿀 수 없다.

0.1 실제 상황으로 이해하기

실제 시나리오

[상황] 중고나라에서 100만원짜리 명품 스니커즈를 파는 사람이 있다.

지금까지는? → "정품이에요"라는 말 외에 믿을 방법이 없다.

→ 가품 논란이 생기면 판매자/구매자 모두 손해.

Attestry가 있다면?

- ① 스니커즈에 붙은 QR을 스마트폰으로 스캔한다.
- ② 화면에 "Brand LV 공식 등록 제품 / 제조일: 2026-02" 가 뜬다.
- ③ "소유권 이전 3회 / 수선 1회 (2026-03, 세탁소 A)" 이력이 보인다.
- ④ 이 기록은 등록된 순간부터 단 한 글자도 수정된 적이 없다.

→ 판매자는 "여기 QR 찍어보세요"로 신뢰를 증명한다.

→ 구매자는 QR 하나로 100만원짜리 판단을 할 수 있다.

0.2 이 서비스에 등장하는 사람들

 브랜드/제조사 서비스 시작점 "이 제품은 우리가 만든 정품입니다"라고 등록. QR 태그를 제품에 부착.	 판매처 (Retail) 중간 판매자 소비자에게 판매 후 클레임 코드 발급. 구매자가 이 코드로 소유권을 가져감.	 소유자 (구매자) 최종 사용자 클레임 코드로 소유권 등록. 수선 승인, 중고 양도, 리셀 모두 직접 함.
 수선업체/세탁소 서비스 제공자	 일반인 (QR 스캔) 공개 조회자	

수선 완료 후 사진과 함께 케이스 제출. 소유자가
승인해야 이력에 기록됨.

앱 없이 QR만 찍으면 공개 이력 조회 가능.
개인정보는 절대 볼 수 없음.

0.3 Attestry의 역할 — 판정이 아닌 "공증"

■ 쉬운 설명

중요한 차이:

✗ Attestry가 하지 않는 것: "이 제품이 정품인지 우리가 심사/판정"

✓ Attestry가 하는 것: "브랜드가 정품이라고 선언한 사실을 변조 불가하게 보관"

비유하자면, 공증 사무소와 같다.

공증 사무소는 계약서 내용이 맞는지 판단하지 않는다.

다만 "이 날, 이 사람이, 이 내용으로 서명했음"을 영구히 증명한다.

브랜드가 선언 → Attestry가 봉인 → 누구도 그 기록을 바꿀 수 없다.

1장. 실물 QR 태그 — "이 QR 안에는 뭐가 들어있나?"

쉬운 설명

QR 코드 안에 들어있는 것은 딱 하나:

QRPUB-550e8400-e29b-41d4-a716-446655440000

또는 URL 형식으로:

<https://attestry.io/p/QRPUB-550e8400-e29b-41d4-a716-446655440000>

이 코드(qrPublicCode)는 제품의 "디지털 주소"다.

스마트폰이 이 주소로 서버에 접속하면 → 서버가 이 제품의 모든 공개 이력을 보여준다.

왜 이렇게 설계했나?

왜 QR 안에 제품 정보를 직접 넣지 않나?

만약 QR 안에 "LV 스니커즈 V1, serial: LV-123..."을 직접 넣으면:

→ 사진 찍어서 똑같이 인쇄하면 복제 완료. 아무 가방에나 붙일 수 있다.

대신 의미 없는 랜덤 UUID 하나만 넣으면:

→ 복사해도 서버에서 "이 UUID에 연결된 실제 이력"은 절대 바꿀 수 없다.

→ 복제 QR을 붙인 가품이 스캔되면, 서버 기록은 여전히 진품의 이력을 가리킨다.

→ 소유권 이전은 현재 소유자 로그인이 필요 → 복제 QR로 소유권 탈취 불가.

1.1 QR 복제를 어떻게 막나?

방어 단계	방법	효과
1단계	UUID v4 (128비트 랜덤) 코드 사용	무작위 대입으로 유효한 코드 추측 불가. 초당 10억 번 시도해도 우주 나이보다 긴 시간 소요.
2단계 (선택)	NFC 칩 + 홀로그램 태그	물리적 복제 자체를 어렵게 만드는 옵션. 브랜드의 선택 사항.

핵심 포인트

핵심: QR을 복제해도 "소유권 탈취"는 불가능하다. 소유권 이전은 반드시 현재 소유자의 로그인 세션이 필요하기 때문입니다.

2장. 절대 원칙 — 이것만 지키면 서비스 신뢰가 산다.

원칙	한 줄 설명	왜 필요한가 ?
① 고정 QR	제품 1개 = QR 1개. 소유자가 바뀌어도, 수선을 해도 QR은 절대 안 바뀐다.	QR이 바뀌면 이미 인쇄된 태그가 전부 무효화. 제품 수명 동안 같은 주소여야 의미가 있다.
② 추가만 가능	원장(이력)에는 새 기록을 추가만 할 수 있고, 기존 기록을 수정/삭제할 수 없다.	수정이 가능하다면 "기록을 믿을 수 있다"는 서비스 핵심 가치가 사라진다.
③ 브랜드 선언 = 정품	브랜드가 등록할 때 디지털 서명으로 "정품 선언"을 한다. Attestry는 이를 영구 보관.	판정 기관 역할을 피함. 책임은 브랜드에게, 증명은 Attestry가 한다.
④ 파일 내용 해시	사진 증거는 파일 내용을 SHA-256으로 해시한다. URL 해시 금지.	URL은 같아도 파일이 바뀔 수 있다. 파일 바이트를 해시해야 내용 변조를 감지할 수 있다.
⑤ 개인정보 없는 공개 조회	QR 스캔 시 이름·이메일·전화번호는 절대 노출 안 됨.	소유권 이전 횟수·이력 등 필요한 공개 정보만 보여준다.
⑥ 토큰 원문 미저장	양도/클레임 코드의 실제 값은 DB에 저장 안 하고, SHA-256 해시만 저장.	DB가 해킹돼도 코드 원문을 역산할 수 없어 소유권 탈취 불가.
⑦ 양도 시 수령 사진 필수	소유권 이전을 확정하려면 반드시 수령 사진을 올려야 한다.	"받지 않았다"는 분쟁 발생 시 증거. 디지털 영수증 역할.

3장. 해시체인 — "왜 기록을 못 바꾸나?"

쉬운 설명

[비유] 해시체인은 "번호 잠금 일기장"이다.

일반 일기장: 어제 쓴 내용을 오늘 지우고 새로 쓸 수 있다.

번호 잠금 일기장 규칙:

- 매 페이지에 "이전 페이지 내용을 기반으로 만든 잠금번호"를 적는다.
- 이전 페이지 내용을 고치면 → 잠금번호가 달라진다.
- 잠금번호가 달라지면 → 그 이후 모든 페이지가 연쇄적으로 틀려진다.
- 따라서 중간 내용을 바꾸면 즉시 들킨다.

Attestry 원장도 똑같다:

seq 1(민팅) → seq 2(출고) → seq 3(판매) → seq 4(클레임) →...

각 기록은 이전 기록의 hash를 포함해서 새 hash를 만든다.

seq 3를 바꾸면 seq 4, 5, 6, 7의 hash가 전부 달라져 → 변조 즉시 탐지.

3.1 해시 계산 방법

```
// — Step 1: 데이터 내용을 해시로 만든다 —
// JCS = JSON Canonicalization Scheme
// → JSON은 키 순서가 구현마다 다를 수 있어서, 항상 같은 순서로 정렬한 뒤 해시.
// → 같은 내용이면 항상 같은 해시값이 나온다는 보장.
data_hash = SHA256(JCS_정규화(data_json))

// — Step 2: 이전 기록과 현재 기록을 묶어서 해시를 만든다 —
// 이전 기록의 hash(prev_hash)를 포함하기 때문에 "체인"이 형성된다.
// 이전 기록을 바꾸면 → prev_hash가 달라지고 → 현재 hash도 달라진다.
hash = SHA256(
  passport_id + seq + event_action +
  actor_role + actor_id + occurred_at +
  correlation_id + prev_hash + data_hash
)

// — seq=1 (첫 번째 기록) —
// 이전 기록이 없으므로 prev_hash = null (Genesis Block)
prev_hash = null // 첫 번째 기록은 연결할 이전이 없으니까
```

왜 이렇게 설계했나?

왜 "낙관적 락(seq 중복 방지)"이 있나? → 실제로는 거의 필요 없다.

물건 1개 = QR 1개 = 원장 1개 구조라서, 동일 제품에 동시 이벤트가 들어올 일이 거의 없다.

다만 만약의 경우(네트워크 재시도 등)에 같은 seq가 두 번 쓰이는 걸 DB UNIQUE 제약으로 막는다.

→ 충돌 시 가장 나중에 들어온 요청이 seq를 다시 받아 재시도. 실제 운영에서 거의 발생 안 함.

→ "안전장치"로 두되, 핵심 기능으로 홍보할 필요는 없다.

4장. DB 테이블 구조 — "어디에 뭘 저장하나?"

쉬운 설명

테이블이 5개다. 각 테이블의 역할:

- ① **assets** → 실물 제품 정보 (변하지 않는 것들: 시리얼, 모델, 브랜드)
- ② **passports** → 디지털 여권 (QR 코드와 연결되는 주인공)
- ③ **passport_owners** → 지금 소유자가 누구인지 빠르게 조회하기 위한 별도 테이블
- ④ **ledger_entries** → 모든 이력 기록 (추가만 가능, 절대 수정/삭제 불가)
- ⑤ **transfer_tokens** → 클레임/양도 코드 (1회용, 만료시간 있음)
- ⑥ **service_cases** → 수선/세탁 케이스
- ⑦ **evidences** → 증거 사진/영수증 파일 정보
- ⑧ **scan_logs** → QR 스캔 기록 (이상 감지용) <- 필요한가~~?

4.1 assets — 실물 제품 (절대 안 바뀌는 정보)

```
// 이 테이블은 "물리 제품의 출생 신고서"다.
// serial_number, model, manufactured_at은 한 번 저장하면 절대 수정 금지.
// 수정이 가능하면 "이 제품이 뭐였는지"를 바꿀 수 있어서 신뢰가 무너진다.
{
  "asset_id": "A100", // 자산 고유 ID (PK)
  "serial_number": "LV-12340-3423-123123", // 시리얼 번호 ★ 절대 수정 금지
  "model_id": "M55", // 모델 ID
  "model_name": "Cyber Sneaker V1", // 모델명
  "brand_id": "ORG_LV", // 브랜드 ID
  "manufactured_at": "2026-02", // 제조년월 ★ 절대 수정 금지

  "asset_state": "ACTIVE", // ACTIVE=정상 / RETIRED=은퇴 /
  VOIDED=무효
  "risk_flag": "NONE", // NONE=정상 / STOLEN=도난 / LOST=분실
  / DISPUTED=분쟁
  // ※ 도난/분실은 VOIDED가 아님! risk_flag로 따로 표시.
  // VOIDED는 "이 제품 자체가 완전 무효"일 때만 사용 (위조 등록, 완전 파기 등)

  // 정품 인증: 브랜드가 선언, Attestry는 보관/증명만
  "is_brand_certified": true, // 브랜드 정품 선언 여부
  "brand_signature": "Base64(sign(assetId|serial|brandId))",
  // brand_signature = 브랜드 개인키로 서명한 값.
  // 누구든 브랜드 공개키로 이 서명을 검증할 수 있다 = 수학적 정품 증명.
  "certified_at": "2026-02-19T01:00:00.000Z",
  "certification_ledger_seq": 1, // MINTED 원장 seq. 이 seq의
  hash가 불변 증거.

  "minted_at": "2026-02-19T01:00:00.000Z",
  "void_reason": null // VOIDED일 때만 사유 기록
}
```


4.2 passports — 디지털 여권 (QR의 주인공)

```
// passport = QR로 열리는 "디지털 여권".
// asset과 1:1, qr_public_code와도 1:1.
// 모든 원장 이벤트는 passport_id를 기준으로 쌓인다.
{
  "passport_id": "P100",
  "asset_id": "A100", // assets 테이블과 연결
  (1:1)
  "qr_public_code": "QRPUB-550e8400-e29b-41d4-a716-446655440000", // ★ 영구 불변
  "qr_status": "ACTIVE", // ACTIVE / SUSPENDED
  "issued_at": "2026-02-19T01:00:00.000Z"
}

// QR이 훼손되면? → 같은 qr_public_code로 이미지만 다시 인쇄.
// 코드 자체는 절대 안 바뀐다.

// QR 이미지 파일은 어디에 저장? → 외부 CDN/스토리지.
// 서버는 Outbox로 "이미지 만들어줘" 요청만 보냄. 코드값은 서버가 생성. 동기식
```

4.3 passport_owners — 현재 소유자 빠른 조회

```
// 이 테이블이 따로 필요한 이유:
// "지금 소유자가 누구야?"를 알려면 원장 전체를 처음부터 다 읽어야 한다.
// 원장이 100개 쌓여있으면 100번 읽어야 함 → 느리고 비효율적.
// → 별도 테이블에 "현재 소유자"만 따로 저장해두면 1번만 조회하면 된다.
//
// ★ 이 테이블은 직접 UPDATE 금지.
// 반드시 CLAIMED / TRANSFER_COMPLETED 이벤트가 확정될 때만 갱신.
{
  "passport_id": "P100",
  "owner_user_id": "ORG_IV", // 최초에는 브랜드가 소유
  "since_at": "2026-02-19T01:00:00.000Z",
  "version": 1 // 소유권 변경될 때마다 +1
}
```

4.4 transfer_tokens — 클레임/양도 코드

```
// 클레임 코드, 양도 코드를 관리하는 테이블.
// ★ 중요: 실제 코드(plain text)는 절대 저장 안 함. SHA-256 해시만 저장.
// 이유: DB가 해킹돼도 코드 원문을 역산할 수 없어서 소유권 탈취 불가.
{
  "transfer_id": "TT-CLAIM-1001",
  "passport_id": "P100",
  "from_user_id": "U_STORE", // 발급자 (판매처)
  "to_user_id": null, // null = 누구든 코드 입력 가능 (open claim)
  "purpose": "CLAIM", // CLAIM(최초 소유) or TRANSFER(양도)
  "token_hash": "SHA256(code_plain)", // 코드 해시만 저장
```

```
"state": "INITIATED",           // INITIATED / COMPLETED / CANCELLED / EXPIRED
"accept_method": "ONE_TIME_CODE", // 코드 입력 방식
"expires_at": "2026-02-26T12:05:00.000Z", // 7일 후 만료
"created_at": "2026-02-19T12:05:00.000Z",
"accepted_at": null,             // 수락 완료 시 기록
"cancelled_at": null            // 취소 시 기록
}
```

5장. 원장에 기록되는 이벤트 전체 목록

이벤트	분류	필수 ?	한 줄 설명
MINTED	LIFECYCLE	✓	브랜드가 제품 등록. 정품 선언 포함. 원장의 첫 번째 기록.
RELEASED	LIFECYCLE	✓	출고 완료. 출고 사진 증거 필수.
SOLD_AT_RETAIL	LIFECYCLE	✓	판매처에서 판매 기록. "왜 이 사람이 클레임했나"의 맥락.
CLAIMED	OWNERSHIP	✓	최초 소유권 확정. 딱 1번만 발생.
SERVICE_CONFIRMED	SERVICE	✓	수선/세탁 소유자 승인 확정.
SERVICE_REJECTED	SERVICE	✓	수선 거절도 기록. 거절 이력이 없으면 분쟁 시 증거 없음.
TRANSFER_COMPLETED	OWNERSHIP	✓	소유권 이전 완료. 수령 사진 필수.
TRANSFER_CANCELLED	OWNERSHIP	✓	양도 취소. 즉시 처리 필수. (만료 대기 시 최대 7일간 탈취 위험)
VOIDED	LIFECYCLE	✓	완전 무효화. 이유 필수. 이후 모든 이벤트 차단.
RETIRED	LIFECYCLE	✓	은퇴. 조회만 가능.

6장. 스니커즈 한 켤레의 일생 — 완전한 원장 시퀀스

📖 실제 시나리오

[시나리오] Cyber Sneaker V1 (A100)

브랜드 **ORG_LV**가 제품을 등록하고 QR 태그를 부착.
 판매처 **U_STORE**에서 구매자 **U_B1**이 구매 → 소유권 클레임.
U_B1이 세탁소 **U_LAUNDRY**에 세탁을 맡기고 승인.
 이후 **U_B1**이 중고거래로 **U_B2**에게 직거래 양도.
 → 이 모든 이력이 원장 **seq 1~7**에 순서대로 기록된다.

seq1 MINTED

행위자: ORG_LV (브랜드)

소유: ORG_LV

원장기록

어떤 일이 일어났나: 브랜드가 제품을 **Attestry**에 등록했다.

```
// ★ assets + passports + passport_owners + ledger_entries 가 동시에 생성.
// 이 4개 테이블 INSERT는 반드시 1개 트랜잭션으로 처리.
// 부분 성공(일부만 저장) 절대 허용 안 됨.
{
  "ledger_id": "L-001",
  "passport_id": "P100",           // 원장의 주인공은 passport_id
  "seq": 1,                       // 첫 번째 기록 = Genesis Block

  "event_category": "LIFECYCLE",
  "event_action": "MINTED",

  "actor_role": "BRAND",
  "actor_id": "ORG_LV",

  "correlation_id": "REQ-20260219-0001", // API 요청 추적 ID (중복 요청 방지)
  "occurred_at": "2026-02-19T01:00:00.000Z",

  "data_json": {
    "assetId": "A100",
    "serialNumber": "LV-12340-3423-123123",
    "brandId": "ORG_LV",
    "qrPublicCode": "QRPUB-550e8400-e29b-41d4-a716-446655440000",
    "isBrandCertified": true,
    "brandSignature": "Base64(sign(assetId|serial|brandId))" // 정품 선언 서명
  },

  "prev_hash": null,              // ★ 첫 번째 기록이라 이전이 없음 (Genesis)
  "data_hash": "SHA256(JCS(data_json))",
  "hash": "SHA256(hash_input)"   // 이 hash가 다음 seq의 prev_hash가 됨
}
```

💡 핵심 포인트

이 시점에 **passport_owners**: { passport_id: P100, owner_user_id: ORG_LV, version: 1 }

seq2

RELEASED

행위자: ORG_LV (브랜드)

소유: ORG_LV

원장기록

어떤 일이 일어났나: 제품을 판매처로 출고했다. 출고 사진을 증거로 첨부.

```
// ★ 사진 증거 없이 출고 불가.
// content_hash = SHA256(file_bytes) - 파일 바이트를 직접 해시.
// URL을 해시하면 안 됨: URL은 같아도 파일이 교체될 수 있어서.
{
  "ledger_id": "L-002", "passport_id": "P100", "seq": 2,
  "event_category": "LIFECYCLE", "event_action": "RELEASED",
  "actor_role": "BRAND", "actor_id": "ORG_LV",
  "occurred_at": "2026-02-19T09:00:00.000Z",

  "data_json": {
    "proof": {
      "evidenceGroupId": "EVG-001",          // 증거 파일 묶음 ID
      "evidenceGroupHash": "SHA256(sort([hash_photo1, hash_photo2]).join())"
      // evidenceGroupHash: 모든 사진의 hash를 정렬 후 합쳐서 다시 hash.
      // 사진 1장이라도 바뀌면 이 hash가 달라진다 = 변조 즉시 탐지.
    },
    "note": "판매처 U_STORE로 출고 완료"
  },

  "prev_hash": "hash_of_L001",              // seq 1의 hash를 여기에 연결
  "data_hash": "SHA256(JCS(data_json))",
  "hash": "SHA256(hash_input)"
}
```

seq3

SOLD_AT_RETAIL

행위자: U_STORE (판매처)

소유: ORG_LV

원장기록

어떤 일이 일어났나: 판매처가 구매자에게 판매했다. 영수증 기록.

```
// ★ 왜 SOLD_AT_RETAIL이 필요한가?
// 다음 seq(CLAIMED)에서 "왜 이 사람이 클레임하는가?"의 근거가 된다.
// 원장 이력의 완결성: 판매 기록 없이 클레임만 있으면 맥락이 사라진다.
{
  "ledger_id": "L-003", "passport_id": "P100", "seq": 3,
  "event_category": "LIFECYCLE", "event_action": "SOLD_AT_RETAIL",
  "actor_role": "RETAIL", "actor_id": "U_STORE",
  "occurred_at": "2026-02-20T14:00:00.000Z",

  "data_json": {
    "storeId": "U_STORE",
    "saleRefHash": "SHA256(R-20260220-0042)",
    // saleRefHash: 영수증 번호를 해시해서 저장.
    // 영수증 원문 저장 금지: 개인정보·내부 거래 정보 노출 위험.
    // 해시로도 "이 영수증과 매칭되는가"는 검증 가능.
    "note": "구매자 U_B1에게 판매 완료"
  },
}
```

```

    "prev_hash": "hash_of_L002",
    "data_hash": "SHA256(JCS(data_json))",
    "hash": "SHA256(hash_input)"
  }

```

쉬운 설명

이 시점에 판매처는 구매자에게 클레임 코드를 발급한다.

구매자가 이 코드를 입력하면 → seq 4 CLAIMED 이벤트가 기록된다.

seq4	CLAIMED	행위자: U_B1 (최초 구매자)	소유: ORG_LV → U_B1 ✨	원장기록
------	---------	--------------------	-----------------------------------	------

어떤 일이 일어났나: 구매자 **U_B1**이 클레임 코드를 입력해서 소유권을 가져갔다.

```

// ★ CLAIMED = 최초 소유권 확정. 이 이벤트는 제품 생애 딱 1번만 발생.
// 이후 소유권이 바뀌면 TRANSFER_COMPLETED로 기록. (CLAIMED 아님)
{
  "ledger_id": "L-004", "passport_id": "P100", "seq": 4,
  "event_category": "OWNERSHIP", "event_action": "CLAIMED",
  "actor_role": "OWNER", "actor_id": "U_B1",
  "correlation_id": "TT-CLAIM-1001", // 어떤 클레임 토큰으로 수락했는지 연결
  "occurred_at": "2026-02-20T15:30:00.000Z",

  "data_json": {
    "claimedBy": "U_B1",
    "claimMethod": "ONE_TIME_CODE",
    "proof": {
      "transferId": "TT-CLAIM-1001",
      "evidenceGroupId": "EVG-002", // 구매 영수증 사진 (선택)
      "evidenceGroupHash": "SHA256(...)"
    }
  },

  "prev_hash": "hash_of_L003",
  "data_hash": "SHA256(JCS(data_json))",
  "hash": "SHA256(hash_input)"
}

// passport_owners 갱신: ORG_LV → U_B1
{ "owner_user_id": "U_B1", "since_at": "2026-02-20T15:30:00.000Z", "version": 2 }

```

seq5	SERVICE_CONFIRMED	행위자: U_B1 (소유자 승인)	소유: U_B1	원장기록
------	-------------------	--------------------	-----------------	------

어떤 일이 일어났나: **U_B1**이 세탁소에 세탁을 맡겼고, 세탁소가 케이스를 제출 → **U_B1**이 승인했다.

```

// ★ 프로세스 순서:

```

```
// 1) 세탁소(U_LAUNDRY)가 service_cases 테이블에 케이스 제출 (SUBMITTED)
// 2) U_B1이 앱에서 승인 또는 거절
// 3) 승인 시 → SERVICE_CONFIRMED 원장 기록
// 4) 거절 시 → SERVICE_REJECTED 원장 기록 (거절도 반드시 기록!)
// ★ 세탁소는 원장에 직접 기록 불가. 소유자 승인 후에만 기록됨.
{
  "ledger_id": "L-005", "passport_id": "P100", "seq": 5,
  "event_category": "SERVICE", "event_action": "SERVICE_CONFIRMED",
  "actor_role": "OWNER", "actor_id": "U_B1", // 승인한 소유자가 actor
  "correlation_id": "SVC-001",
  "occurred_at": "2026-03-05T20:00:00.000Z",

  "data_json": {
    "serviceCaseId": "SVC-001",
    "kind": "CLEANING", // CLEANING / REPAIR / INSPECTION
    "providerId": "U_LAUNDRY", // 세탁소 ID
    "receiptNo": "CL-99121",
    "proof": {
      "evidenceGroupId": "EVG-003", // 세탁 전/후 사진, 영수증
      "evidenceGroupHash": "SHA256(sort([before_hash, after_hash,
receipt_hash]).join())"
    },
    "approvedBy": "U_B1",
    "approvedAt": "2026-03-05T20:00:00.000Z"
  },

  "prev_hash": "hash_of_L004",
  "data_hash": "SHA256(JCS(data_json))",
  "hash": "SHA256(hash_input)"
}
```

쉬운 설명 (비개발자용)

공개 조회 시 수선 이력: "2026-03-05 세탁 완료 (증거 사진 있음)"
 소유자 이름은 절대 노출 안 됨. "승인됨"이라는 사실만 공개.

seq6 TRANSFER_COMPLE TED 행위자: SYSTEM (자동 확정)

소유: U_B1 → U_B2 ✨

원장기록

어떤 일이 일어났나: U_B1이 중고거래로 U_B2에게 직거래 양도했다. 수령 사진 업로드 후 확정.

```
// ★ 직거래 양도 프로세스:
// 1) U_B1이 "양도 시작" → transfer_tokens 생성 (IN_PERSON_QR, TTL 15분)
// 2) U_B2가 현장에서 QR 스캔 → 수령 사진 업로드
// 3) 사진 없으면 거부. 사진 있으면 TRANSFER_COMPLETED 원장 기록.
// 4) actor_role = SYSTEM: 서버가 토큰 검증 후 자동 확정. 사용자가 직접 원장 조작 불가.
{
  "ledger_id": "L-006", "passport_id": "P100", "seq": 6,
  "event_category": "OWNERSHIP", "event_action": "TRANSFER_COMPLETED",
  "actor_role": "SYSTEM", // 서버가 토큰 검증 후 자동 기록
  "actor_id": "SYSTEM",
  "correlation_id": "TT-TRANS-001", // 어떤 양도 토큰으로 완료됐는지
}
```

```

"occurred_at": "2026-03-10T11:00:00.000Z",

"data_json": {
  "transferId": "TT-TRANS-001",
  "fromOwnerRef": "SUBJECT_U_B1",    // ★ 이름/이메일 아닌 내부 참조 ID만
  "toOwnerRef": "SUBJECT_U_B2",    // 개인정보(PII) 원장 기록 절대 금지
  "acceptMethod": "IN_PERSON_QR",    // 직거래 QR 스캔
  "proof": {
    "evidenceGroupId": "EVG-004", // 수령 현장 사진
    "evidenceGroupHash": "SHA256(...)" // 사진 변조 감지용
  },
  "acceptedAt": "2026-03-10T11:00:00.000Z"
},

"prev_hash": "hash_of_L005",
"data_hash": "SHA256(JCS(data_json))",
"hash": "SHA256(hash_input)"
}

// passport_owners 갱신: U_B1(v2) → U_B2(v3)
{ "owner_user_id": "U_B2", "since_at": "2026-03-10T11:00:00.000Z", "version": 3 }

// ★ qr_public_code 변경 없음! 같은 QR로 계속 조회 가능.

```

seq7

TRANSFER_COMPLE
TED

행위자: SYSTEM (자동 확정)

소유: U_B2 → U_B3 ✨

원장기록

어떤 일이 일어났다: **U_B2**가 온라인 중고거래로 **U_B3**에게 리셀했다. 일회성 코드 방식.

```

// ★ 온라인 양도 프로세스:
// 1) U_B2가 "양도 시작" → transfer_tokens 생성 (ONE_TIME_CODE, TTL 7일)
// 2) U_B2가 중고거래 앱 채팅으로 코드를 U_B3에게 전달
// 3) U_B3가 택배 수령 후 앱에서 코드 입력 + 수령 사진 업로드
// 4) 사진 없으면 거부. 사진 있으면 TRANSFER_COMPLETED 원장 기록.
{
  "ledger_id": "L-007", "passport_id": "P100", "seq": 7,
  "event_category": "OWNERSHIP", "event_action": "TRANSFER_COMPLETED",
  "actor_role": "SYSTEM",
  "actor_id": "SYSTEM",
  "correlation_id": "TT-TRANS-002",
  "occurred_at": "2026-03-20T14:30:00.000Z",

  "data_json": {
    "transferId": "TT-TRANS-002",
    "fromOwnerRef": "SUBJECT_U_B2",
    "toOwnerRef": "SUBJECT_U_B3",
    "acceptMethod": "ONE_TIME_CODE", // 온라인 코드 입력
    "proof": {
      "evidenceGroupId": "EVG-005", // 택배 수령 후 언박싱 사진
      "evidenceGroupHash": "SHA256(...)"
    },
    "acceptedAt": "2026-03-20T14:30:00.000Z"
  },
}

```



```

"prev_hash": "hash_of_L006",
"data_hash": "SHA256(JCS(data_json))",
"hash": "SHA256(hash_input)"
}

// passport_owners 갱신: U_B2(v3) → U_B3(v4)
{ "owner_user_id": "U_B3", "since_at": "2026-03-20T14:30:00.000Z", "version": 4 }

```

전체 원장 흐름 한눈에 보기

seq	이벤트	누가	소유자 변화	이 시점에 제품 상태
1	MINTED	브랜드 ORG_LV	ORG_LV (최초)	등록 완료. QR 태그 준비.
2	RELEASED	브랜드 ORG_LV	ORG_LV	출고 완료. 사진 2장 증거.
3	SOLD_AT_RETAIL	판매처 U_STORE	ORG_LV	판매처에서 U_B1에게 판매.
4	CLAIMED	구매자 U_B1	ORG_LV → U_B1	U_B1 소유권 확정.
5	SERVICE_CONFIRMED	소유자 U_B1	U_B1	세탁소 세탁. U_B1 승인.
6	TRANSFER_COMPLETED	SYSTEM	U_B1 → U_B2	직거래 양도. 수령사진 첨부.
7	TRANSFER_COMPLETED	SYSTEM	U_B2 → U_B3	온라인 리셀. 수령사진 첨부.

7장. QR 스캔 시 사용자에게 보이는 화면

■ 쉬운 설명 (비개발자용)

누구든 QR을 스캔하면 이 API가 호출된다:

GET <https://attestry.io/p/QR PUB-550e8400-e29b-41d4-a716-446655440000>

로그인 없이 접근 가능. 하지만 개인정보는 절대 보이지 않는다.

```
// seq 7까지 진행된 시점에서 QR 스캔 시 응답 (현재 소유자: U_B3)
{
  "qr_public_code": "QR PUB-550e8400-e29b-41d4-a716-446655440000",

  "asset": {
    "model_name": "Cyber Sneaker V1",
    "brand_name": "Brand LV",
    "manufactured_at": "2026-02",
    "asset_state": "ACTIVE",           // 정상 상태
    "risk_flag": "NONE"               // 도난/분실 없음
    // ★ serial_number 공개 금지: 복제 공격에 악용될 수 있음
  },

  "certification": {
    "is_brand_certified": true,
    "brand_name": "Brand LV",
    "certified_at": "2026-02-19T01:00:00.000Z",
    "signature_valid": true           // 서버가 brand_signature 검증한 결과
  },

  "ownership": {
    // ★ 소유자 이름/이메일/전화 절대 포함 금지
    "transfer_count": 3,              // 소유권 이전 횟수
    "current_since": "2026-03-20T14:30:00.000Z"
  },

  "service_history": [
    {
      "kind": "CLEANING",             // 세탁
      "confirmed_at": "2026-03-05T20:00:00.000Z",
      "provider": "세탁소 A",         // 업체명만 공개 (개인정보 아님)
      "has_evidence": true            // 사진 증거 있음
    }
  ],

  "ledger_summary": {
    "total_events": 7,
    "latest_event": "TRANSFER_COMPLETED",
    "latest_at": "2026-03-20T14:30:00.000Z"
  }
}
```


8장. 면접에서 나올 질문 & 답변 가이드

질문	핵심 답변
"왜 블록체인 안 썼나요?"	블록체인의 핵심 가치인 "탈중앙화"는 여러 기관이 서로 신뢰할 수 없을 때 필요합니다. Attestry 는 단일 플랫폼이 운영하는 서비스라서 해시체인으로 동일한 번조 불가 보장을 더 빠르고 저렴하게 구현할 수 있었습니다.
"append-only 원장이 뭔가요?"	한번 기록된 데이터는 수정/삭제가 불가능하고 추가만 됩니다. 오류가 있어도 새 이벤트(CORRECTION)로 정정합니다. 이 구조 덕분에 "기록을 바꿀 수 없다"는 신뢰가 생깁니다.
"해시체인이 어떻게 번조를 막나요?"	각 기록은 이전 기록의 hash 를 포함해서 자신의 hash 를 만듭니다. 중간 기록을 바꾸면 그 이후 모든 hash 가 연쇄적으로 틀려지므로 번조가 즉시 탐지됩니다.
"왜 토큰 원문을 저장 안 하나요?"	DB 가 해킹돼도 코드 원문을 역산할 수 없게 하기 위해서입니다. SHA-256 해시만 저장하면 해커가 해시값을 가져가도 원래 코드를 알아낼 수 없어서 소유권 탈취가 불가능합니다.
"동시성 처리를 어떻게 했나요?"	물건 1개 = QR 1개 구조라서 동일 제품에 동일 이벤트가 들어올 일이 거의 없습니다. 다만 DB UNIQUE 제약(passport_id , seq)으로 중복 INSERT 를 막아두었습니다. 실제 운영에서는 거의 발생하지 않는 안전장치입니다.
"EU DPP 규정과 관련이 있나요?"	EU ESPR (친환경 제품 설계 규정)은 2027년부터 주요 제품군에 디지털 제품 여권(DPP)을 의무화합니다. Attestry 는 고정 식별자(QR), 이력 추적, 공개 조회라는 핵심 요건을 갖추고 있습니다. 구체적인 데이터 표준은 아직 확정 중입니다.